CS-201 COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE 4 (3+1)

FINAL SEMESTER PROJECT

Submitted to :
MAM FATIMA RAUF

Submitted by:

EHRAM AYLIA AWAN	**22-cs-027**

Section (c)
Date : 31/05/2024

Department of Computer Science
HITEC University, TAXILA

STATEMENT:

Multi-Purpose Utility Application: Binary Converter, Arithmetic Calculator, Logical Operator, and Student Registration System with Real-Time System Time Calculation.

INTRODUCTION:

This assembly language program is a multi-purpose utility application that includes four main functionalities: binary conversion, arithmetic calculator, logical operator, and student registration system.

Methodology:

The program follows a modular approach where each functionality is implemented as a separate section. The main menu allows the user to choose which operation they want to perform. Depending on the user's choice, the program branches to the corresponding section. Here's how each part works:

1. **Main Menu**:
    - Displays options for the user to choose from.
    - Reads the user's choice and directs them to the corresponding function.
2. **Arithmetic Operations**:
    - Displays a sub-menu with options for addition, subtraction, multiplication, and division.
    - Reads two numbers from the user and performs the chosen arithmetic operation.
    - Displays the result and returns to the main menu.
3. **Logical Operations**:
    - Reads two numbers and a logical operation choice (AND, OR, XOR, NOT) from the user.
    - Performs the chosen logical operation on the numbers.
    - Displays the result and returns to the main menu.
4. **Binary Conversion**:
    - Reads a decimal number from the user.
    - Converts the decimal number to its binary equivalent.

- Displays the binary number and returns to the main menu.

5. **Student Registration System**:
    - Prompts the user to enter student details (name, registration number, program, semester, CGPA).
    - Stores and then displays the entered details.
    - Returns to the main menu.

CODE:

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
include 'emu8086.inc'
.MODEL SMALL
.STACK 100H
.DATA

;arithmetic operations
msg1 DB 'For Add type    :1$'
msg2 DB 10,13,'For Sub type    :2$'
msg3 DB 10,13,'For Mul type    :3$'
msg4 DB 10,13,'For Div type    :4$'
msg_ DB 10,13,'For Exit from this portion   :5$'
msg5 DB 10,13,'Choose Any One:$'
msg6 DB 10,13,10,13,'Enter 1st Number:$'
msg7 DB 10,13,'Enter 2nd Number:$'
msg8 DB 10,13,10,13,'The Result is:$'


num1 DB ?
num2 DB ?
num3 DB ?
num4 DB ?
num5 DB ?
num6 DB ?
num7 DB ?
num8 DB ?
result DB ?
remainder DB ?
quotient DB ?

;logical operations
msg9 db "invalid operation selected $ "
msg10 db 10,13,"Enter the first number: $"
msg11 db 10,13,"Enter the second number: $"
msg12 db 10,13,"Select operation (1: AND, 2: OR, 3: XOR, 4: NOT): $"
result_msg db 10,13,"Result: $"
newline db 0dh, 0ah, "$"
input1 db ?
input2 db ?
operation db ?

;binary conversion
msg13 db "Enter the decimal number: $"
msg14 db 0dh, 0ah, "Binary number: $"
num9 dw ?

;student system
m1 db 100 dup('$')
m2 db 100 dup('$')
m3 db 100 dup('$')
m4 db 100 dup('$')
m5 db 100 dup('$')

.code
main proc

    mov ax, @data
    mov ds, ax

```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    mov ax, @data
    mov ds, ax

    main_menu:
    mov ah, 9
    lea dx, newline
    int 21h

    print '                    ***____WELCOME TO THE COAL PROJECT____***'
    mov ah, 9
    lea dx, newline
    int 21h
    mov ah, 9
    lea dx, newline
    int 21h

    print 'ENTER 1 FOR ARITHMETIC OPERATIONS '
    mov ah, 9
    lea dx, newline
    int 21h

    PRINT 'ENTER 2 FOR LOGICAL OPERATIONS '
    mov ah, 9
    lea dx, newline
    int 21h

    PRINT 'ENTER 3 FOR BINAZRY CONVERSION '
    mov ah, 9
    lea dx, newline
    int 21h

    PRINT 'ENTER 4 FOR STUDENT RECORD SYSTEM '
    mov ah, 9
    lea dx, newline
    int 21h

    PRINT 'ENTER 5 TO EXIT '
    mov ah, 9
    lea dx, newline
    int 21h


    mov ah, 1
    int 21h
    sub al, '0'
    mov operation, al

    mov ah, 9
    lea dx, newline
    int 21h

    cmp operation, 1
    je arithmetic_operations
    cmp operation, 2
    je logical_operations
    cmp operation, 3
    je binary_conversion
    cmp operation, 4
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    je binary_conversion
    cmp operation, 4
    je student_record_system
    cmp operation,5
    je exitcode
    jmp main_menu

arithmetic_operations:
ask:
    lea dx, msg1
    mov ah, 9
    int 21h

    lea dx, msg2
    mov ah, 9
    int 21h

    lea dx, msg3
    mov ah, 9
    int 21h

    lea dx, msg4
    mov ah, 9
    int 21h

    lea dx, msg_
    mov ah, 9
    int 21h

    lea dx, msg5
    mov ah, 9
    int 21h

    mov ah, 1
    int 21h
    sub al, 30h
    mov bh, al

    cmp bh, 1
    je add

    cmp bh, 2
    je sub

    cmp bh, 3
    je mul

    cmp bh, 4
    je div

    cmp bh, 5
    je exit

    jmp ask

add:
    lea dx, msg6
    mov ah, 9
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    mov ah, 9
    int 21h

    mov ah, 1
    int 21h
    sub al, 30h
    mov num1, al

    lea dx, msg7
    mov ah, 9
    int 21h

    mov ah, 1
    int 21h
    sub al, 30h
    mov num2, al

    mov al, num1
    add al, num2
    mov ah, 0
    aaa

    mov bh, ah
    mov bl, al

    lea dx, msg8
    mov ah, 9
    int 21h

    mov dl, bh
    add dl, 48
    mov ah, 2
    int 21h

    mov dl, bl
    add dl, 48
    mov ah, 2
    int 21h

    jmp space

sub:
    lea dx, msg6
    mov ah, 9
    int 21h

    mov ah, 1
    int 21h
    sub al, 30h
    mov num3, al
    mov bl,num3

    lea dx, msg7
    mov ah, 9
    int 21h

    mov ah, 1
    int 21h
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    mov ah, 1
    int 21h
    sub al, 30h
    mov num4, al

    sub bl, num4

    lea dx, msg8
    mov ah, 9
    int 21h

    mov dl,bl
    add dl,30h
    mov ah, 2
    int 21h

    jmp space

mul:
    lea dx, msg6
    mov ah, 9
    int 21h

    mov ah, 1
    int 21h
    sub al, 30h
    mov num5, al

    lea dx, msg7
    mov ah, 9
    int 21h

    mov ah, 1
    int 21h
    sub al, 30h
    mov num6, al

    mov al, num5
    mov bl, num6
    mul bl

    aam

    mov ch, ah
    mov cl, al

    lea dx, msg8
    mov ah, 9
    int 21h

    mov dl, ch
    add dl, 48
    mov ah, 2
    int 21h

    mov dl, cl
    add dl, 48
    mov ah, 2
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    mov dl, cl
    add dl, 48
    mov ah, 2
    int 21h

    jmp space

div:
    lea dx, msg6   ;
    mov ah, 9
    int 21h

    mov ah, 1   ; read dividend (divided)
    int 21h
    sub al, 30h
    mov num7, al

    lea dx, msg7
    mov ah, 9
    int 21h

    mov ah, 1   ; read divisor (dividing)
    int 21h
    sub al, 30h
    mov num8, al

    cmp num8, 0
    je div_error

    mov al, num7
    mov ah, 0   ; clear AH for division
    div num8

    mov quotient, ah
    mov remainder, al

    lea dx, msg8
    mov ah, 9
    int 21h

    mov dl, quotient
    add dl, 48
    mov ah, 2
    int 21h

    mov dl, remainder
    add dl, 48
    mov ah, 2
    int 21h

    jmp space

div_error:
    lea dx, msg8
    mov ah, 9
    int 21h

    jmp space
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    jmp space

exit:

    jmp main_menu

space:
    lea dx, msg5
    mov ah, 9
    int 21h

    mov dl, 0dh  ;left most13
    mov ah, 2
    int 21h

    mov dl, 0ah  ;new line10
    mov ah, 2
    int 21h

    jmp ask

logical_operations:
    mov ah, 9
    lea dx, msg10
    int 21h

    mov ah, 1
    int 21h
    sub al, '0'
    mov input1, al

    mov ah, 9
    lea dx, msg11
    int 21h

    mov ah, 1
    int 21h
    sub al, '0'
    mov input2, al

    mov ah, 9
    lea dx, msg12
    int 21h

    mov ah, 1
    int 21h
    sub al, '0'
    mov operation, al


    mov ah, 9
    lea dx, result_msg
    int 21h

    cmp operation, 1
    je perform_and
    cmp operation, 2
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    je perform_and
    cmp operation, 2
    je perform_or
    cmp operation, 3
    je perform_xor
    cmp operation, 4
    je perform_not

    mov ah, 9
    lea dx, newline
    int 21h
    mov ah, 9
    lea dx, msg9
    int 21h
    jmp exit_program

perform_and:
    mov al, input1
    and al, input2
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
    jmp exit_program

perform_or:
    mov al, input1
    or al, input2
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
    jmp exit_program

perform_xor:
    mov al, input1
    xor al, input2
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
    jmp exit_program

perform_not:
    mov al, input1
    not al
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
    jmp exit_program

exit_program:

    jmp main_menu

binary_conversion:
    mov ah, 9
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
binary_conversion:
    mov ah, 9
    lea dx, msg13
    int 21h

    mov ah, 1
    int 21h

    sub al, 30h
    mov ah,0
    mov num9, ax

    mov bx, 2
    mov cx, 0
    mov dx,0

    again:
    mov ax, num9
    div bx
    mov num9, ax
    push dx
    inc cx
    cmp ax, 0
    jne again

    mov ah, 9
    lea dx, msg14
    int 21h

    display:
    pop dx
    add dl, '0'
    mov ah, 2
    int 21h
    loop display

jmp main_menu

student_record_system:

    print '___THIS IS A STUDENT RECORD SYSTEM___ '
    mov ah, 9
    lea dx, newline
    int 21h

    print 'enter name of the student  :  '
    mov si, offset m1
    call gotoloop

    mov ah, 9
    lea dx, newline
    int 21h

    print 'enter registration number of the student :  '
    mov si, offset m2
    call gotoloop

    mov ah, 9
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    mov ah, 9
    lea dx, newline
    int 21h

    print 'enter program of the student : '
    mov si,offset m3
    call gotoloop

    mov ah, 9
    lea dx, newline
    int 21h

    print 'enter current semester of the student : '
    mov si,offset m4
    call gotoloop

    mov ah, 9
    lea dx, newline
    int 21h

    print 'enter current cgpa of the student : '
    mov si,offset m5
    call gotoloop

    mov ah, 9
    lea dx, newline
    int 21h

    print 'NAME  :'
    mov ah, 9
    lea dx, m1
    int 21h

    mov ah, 9
    lea dx, newline
    int 21h

    print 'REG NO. :'
    mov ah, 9
    lea dx, m2
    int 21h

    mov ah, 9
    lea dx, newline
    int 21h

    print 'PROGRAM :'
    mov ah,9
    lea dx , m3
    int 21h

    mov ah, 9
    lea dx, newline
    int 21h

    print 'CURRENT SEMESTER :'
    mov ah,9
```

edit: C:\emu8086\MySource\coal1.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
    print 'CURRENT SEMESTER :'
    mov ah,9
    lea dx , m4
    int 21h

    mov ah, 9
    lea dx, newline
    int 21h

    print 'CURRENT CGPA :'
    mov ah,9
    lea dx , m5
    int 21h

    mov ah, 9
    lea dx, newline
    int 21h

    jmp main_menu

    exitcode:

    print 'THANK YOU'
    mov ah, 9
    lea dx, newline
    int 21h

    print 'GOOD BYE'
    mov ah, 9
    lea dx, newline
    int 21h


     mov ah,4ch
     int 21h

     space1:
    lea dx, msg5
    mov ah, 9
    int 21h

    mov dl, 0dh
    mov ah, 2
    int 21h

    mov dl, 0ah
    mov ah, 2
    int 21h


main endp

gotoloop proc
l1:
    mov ah, 1
    int 21h
    cmp al, 13
    je end_input
```

```
main endp

gotoloop proc
l1:
    mov ah, 1
    int 21h
    cmp al, 13
    je end_input
    mov [si], al
    inc si
    jmp l1

end_input:
    mov byte ptr [si], '$'
    ret
gotoloop endp

end main
```

CODE TEXT:

```
include 'emu8086.inc'

.MODEL SMALL

.STACK 100H

.DATA


;arithmetic operations

msg1 DB 'For Add type   :1$'

msg2 DB 10,13,'For Sub type   :2$'

msg3 DB 10,13,'For Mul type   :3$'

msg4 DB 10,13,'For Div type   :4$'

msg_ DB 10,13,'For Exit from this portion  :5$'

msg5 DB 10,13,'Choose Any One:$'

msg6 DB 10,13,10,13,'Enter 1st Number:$'

msg7 DB 10,13,'Enter 2nd Number:$'

msg8 DB 10,13,10,13,'The Result is:$'
```

```
num1 DB ?

num2 DB ?

num3 DB ?

num4 DB ?

num5 DB ?

num6 DB ?

num7 DB ?

num8 DB ?

result DB ?

remainder DB ?

quotient DB ?


;logical operations

msg9 db "invalid operation selected $ "

msg10 db 10,13,"Enter the first number: $"

msg11 db 10,13,"Enter the second number: $"

msg12 db 10,13,"Select operation (1: AND, 2: OR, 3: XOR, 4: NOT): $"

result_msg db 10,13,"Result: $"

newline db 0dh, 0ah, "$"

input1 db ?

input2 db ?

operation db ?
```

```
;binary conversion
msg13 db "Enter the decimal number: $"
msg14 db 0dh, 0ah, "Binary number: $"
num9 dw ?

;student system
m1 db 100 dup('$')
m2 db 100 dup('$')
m3 db 100 dup('$')
m4 db 100 dup('$')
m5 db 100 dup('$')

.code
main proc
   mov ax, @data
   mov ds, ax
   main_menu:
   mov ah, 9
   lea dx, newline
   int 21h
   print '            ***____WELCOME TO THE COAL PROJECT____***'
   mov ah, 9
   lea dx, newline
   int 21h
```

```
mov ah, 9
lea dx, newline
int 21h
print 'ENTER 1 FOR ARITHMETIC OPERATIONS '
mov ah, 9
lea dx, newline
int 21h
PRINT 'ENTER 2 FOR LOGICAL OPERATIONS '
mov ah, 9
lea dx, newline
int 21h
PRINT 'ENTER 3 FOR BINAZRY CONVERSION '
mov ah, 9
lea dx, newline
int 21h
PRINT 'ENTER 4 FOR STUDENT RECORD SYSTEM '
mov ah, 9
lea dx, newline
int 21h

PRINT 'ENTER 5 TO EXIT '
mov ah, 9
lea dx, newline
int 21h
```

```
    mov ah, 1
    int 21h
    sub al, '0'
    mov operation, al
      mov ah, 9
    lea dx, newline
    int 21h
    cmp operation, 1
    je arithmetic_operations
    cmp operation, 2
    je logical_operations
    cmp operation, 3
    je binary_conversion
    cmp operation, 4
    je student_record_system
    cmp operation,5
    je exitcode
    jmp main_menu
arithmetic_operations:
ask:
    lea dx, msg1
    mov ah, 9
    int 21h
    lea dx, msg2
```

```
mov ah, 9
int 21h
lea dx, msg3
mov ah, 9
int 21h
lea dx, msg4
mov ah, 9
int 21h
lea dx, msg_
mov ah, 9
int 21h
lea dx, msg5
mov ah, 9
int 21h
mov ah, 1
int 21h
sub al, 30h
mov bh, al
cmp bh, 1
je add
cmp bh, 2
je sub
cmp bh, 3
je mul
```

```
    cmp bh, 4
    je div
    cmp bh, 5
    je exit
    jmp ask
add:
    lea dx, msg6
    mov ah, 9
    int 21h
    mov ah, 1
    int 21h
    sub al, 30h
    mov num1, al
    lea dx, msg7
    mov ah, 9
    int 21h
    mov ah, 1
    int 21h
    sub al, 30h
    mov num2, al
    mov al, num1
    add al, num2
    mov ah, 0
    aaa
```

```
    mov bh, ah
    mov bl, al
    lea dx, msg8
    mov ah, 9
    int 21h
    mov dl, bh
    add dl, 48
    mov ah, 2
    int 21h
     mov dl, bl
    add dl, 48
    mov ah, 2
    int 21h
    jmp space

sub:
    lea dx, msg6
    mov ah, 9
    int 21h
    mov ah, 1
    int 21h
    sub al, 30h
    mov num3, al
    mov bl,num3
```

```
    lea dx, msg7
    mov ah, 9
    int 21h
   mov ah, 1
    int 21h
    sub al, 30h
    mov num4, al
    sub bl, num4
      lea dx, msg8
    mov ah, 9
    int 21h
    mov dl,bl
    add dl,30h
    mov ah, 2
    int 21h
    jmp space

mul:
    lea dx, msg6
    mov ah, 9
    int 21h
    mov ah, 1
    int 21h
    sub al, 30h
```

```
mov num5, al
lea dx, msg7
mov ah, 9
int 21h

mov ah, 1
int 21h
sub al, 30h
mov num6, al
mov al, num5
mov bl, num6
mul bl
aam
mov ch, ah
mov cl, al
lea dx, msg8
mov ah, 9
int 21h
mov dl, ch
add dl, 48
mov ah, 2
int 21h
mov dl, cl
add dl, 48
```

```
    mov ah, 2
    int 21h
    jmp space
div:
    lea dx, msg6  ;
    mov ah, 9
    int 21h
    mov ah, 1  ; read dividend (divided)
    int 21h
    sub al, 30h
    mov num7, al
    lea dx, msg7
    mov ah, 9
    int 21h
    mov ah, 1  ; read divisor (dividing)
    int 21h
    sub al, 30h
    mov num8, al
    cmp num8, 0
    je div_error
    mov al, num7
    mov ah, 0  ; clear AH for division
    div num8
    mov quotient, ah
```

```
    mov remainder, al
    lea dx, msg8
    mov ah, 9
    int 21h
    mov dl, quotient
    add dl, 48
    mov ah, 2
    int 21h
    mov dl, remainder
    add dl, 48
    mov ah, 2
    int 21h

    jmp space
div_error:
    lea dx, msg8
    mov ah, 9
    int 21h
    jmp space
exit:
    jmp main_menu
space:
    lea dx, msg5
    mov ah, 9
```

```
    int 21h
    mov dl, 0dh  ;left most13
    mov ah, 2
    int 21h
    mov dl, 0ah  ;new line10
    mov ah, 2
    int 21h
    jmp ask
logical_operations:
    mov ah, 9
    lea dx, msg10
    int 21h
    mov ah, 1
    int 21h
    sub al, '0'
    mov input1, al
    mov ah, 9
    lea dx, msg11
    int 21h
    mov ah, 1
    int 21h
    sub al, '0'
    mov input2, al
    mov ah, 9
```

```
lea dx, msg12
int 21h
mov ah, 1
int 21h
sub al, '0'
mov operation, al
mov ah, 9
lea dx, result_msg
int 21h
cmp operation, 1
je perform_and
cmp operation, 2
je perform_or
cmp operation, 3
je perform_xor
cmp operation, 4
je perform_not

mov ah, 9
lea dx, newline
int 21h
mov ah, 9
lea dx, msg9
int 21h
```

```
    jmp exit_program
perform_and:
    mov al, input1
    and al, input2
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
    jmp exit_program
perform_or:
    mov al, input1
    or al, input2
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
    jmp exit_program
perform_xor:
    mov al, input1
    xor al, input2
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
```

```
    jmp exit_program
perform_not:
    mov al, input1
    not al
    add al, '0'
    mov dl, al
    mov ah, 2
    int 21h
    jmp exit_program
exit_program:
    jmp main_menu
binary_conversion:
    mov ah, 9
    lea dx, msg13
    int 21h
    mov ah, 1
    int 21h
    sub al, 30h
    mov ah,0
    mov num9, ax
    mov bx, 2
    mov cx, 0
    mov dx,0
    again:
```

```
    mov ax, num9
    div bx
    mov num9, ax
    push dx
    inc cx
    cmp ax, 0
    jne again

    mov ah, 9
    lea dx, msg14
    int 21h

    display:
    pop dx
    add dl, '0'
    mov ah, 2
    int 21h
    loop display
jmp main_menu
student_record_system:
    print '___THIS IS A STUDENT RECORD SYSTEM___ '
    mov ah, 9
    lea dx, newline
    int 21h
```

```
print 'enter name of the student  :  '
mov si, offset m1
call gotoloop
mov ah, 9
lea dx, newline
int 21h
print 'enter registration number of the student :  '
mov si, offset m2
call gotoloop
mov ah, 9
lea dx, newline
int 21h
print 'enter program of the student : '
mov si,offset m3
call gotoloop
mov ah, 9
lea dx, newline
int 21h
print 'enter current semester of the student : '
mov si,offset m4
call gotoloop
mov ah, 9
lea dx, newline
int 21h
```

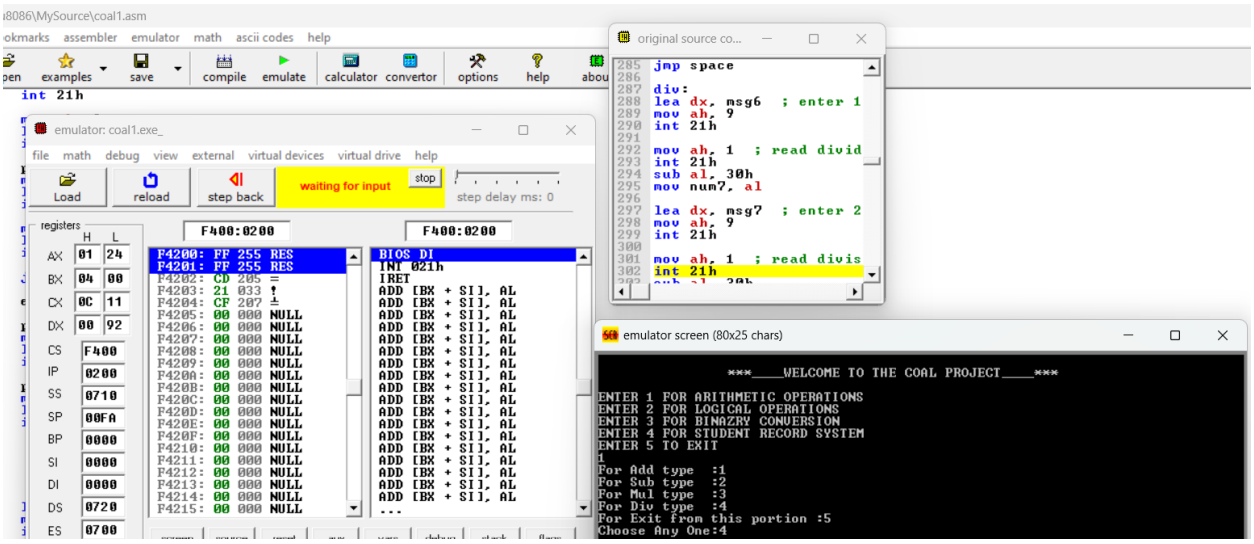
```
print 'enter current cgpa of the student : '
mov si,offset m5
call gotoloop
mov ah, 9
lea dx, newline
int 21h
print 'NAME  :'
mov ah, 9
lea dx, m1
int 21h
mov ah, 9
lea dx, newline
int 21h
print 'REG NO. :'
mov ah, 9
lea dx, m2
int 21h
mov ah, 9
lea dx, newline
int 21h
print 'PROGRAM :'
mov ah,9
lea dx , m3
int 21h
```

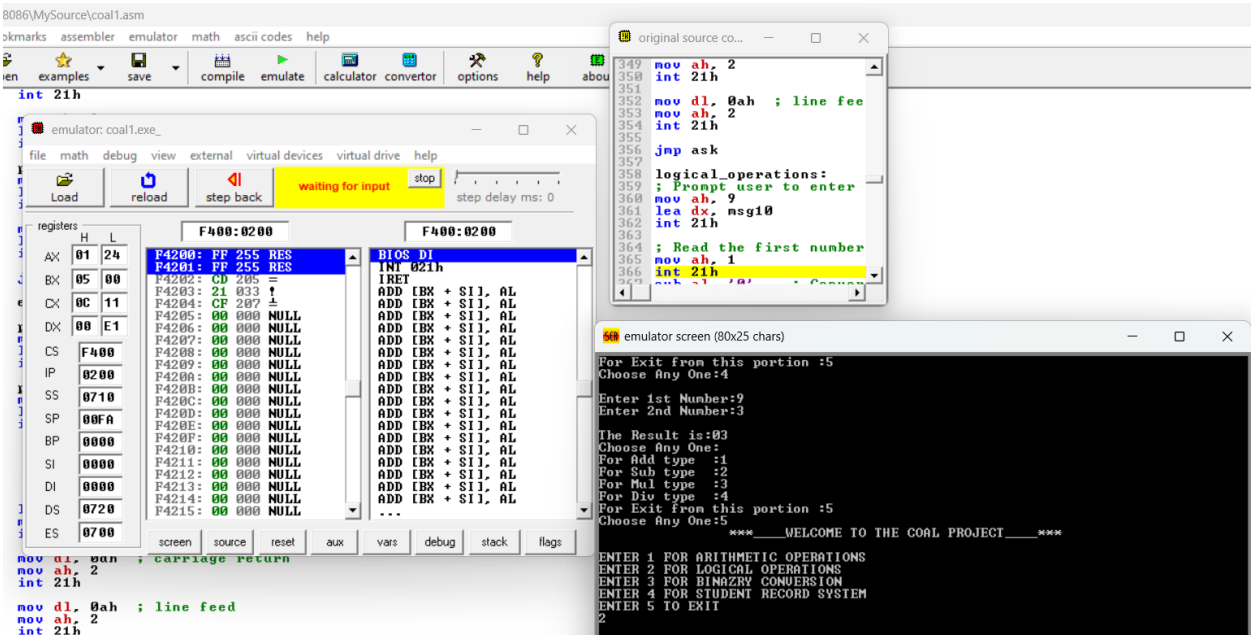
```
mov ah, 9
lea dx, newline
int 21h
print 'CURRENT SEMESTER :'
mov ah,9
lea dx , m4
int 21h
mov ah, 9
lea dx, newline
int 21h
print 'CURRENT CGPA :'
mov ah,9
lea dx , m5
int 21h
mov ah, 9
lea dx, newline
int 21h
jmp main_menu
exitcode:
print 'THANK YOU'
mov ah, 9
lea dx, newline
int 21h
print 'GOOD BYE'
```

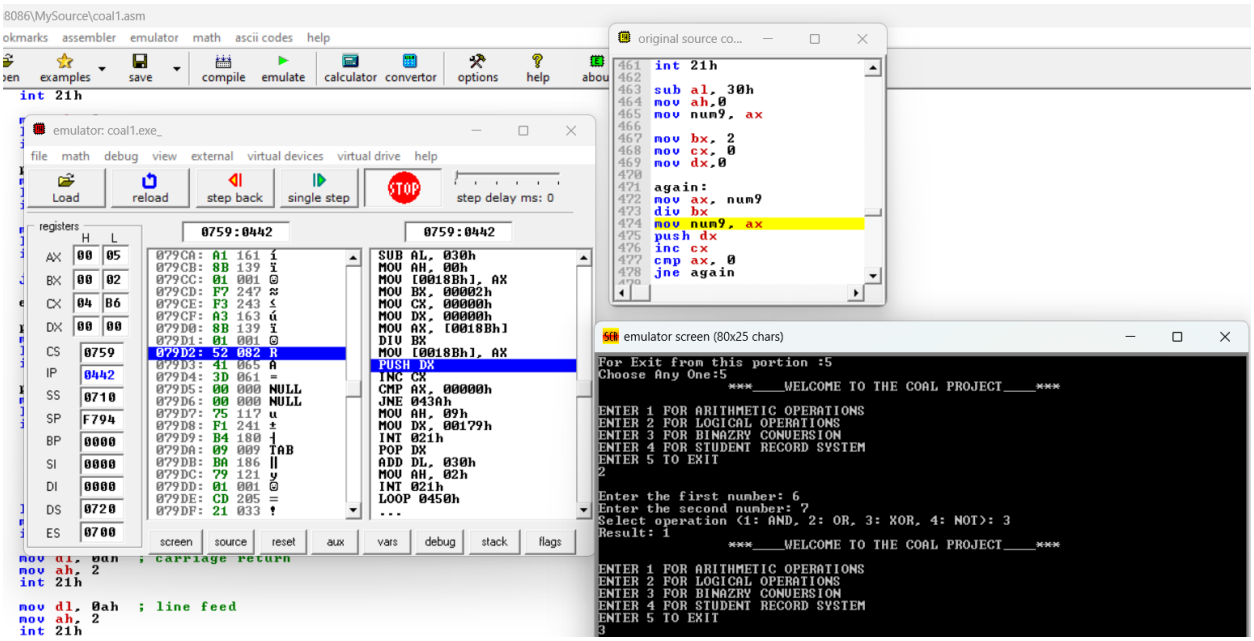
```
    mov ah, 9
    lea dx, newline
    int 21h
   mov ah,4ch
     int 21h
     space1:
    lea dx, msg5
    mov ah, 9
    int 21h
    mov dl, 0dh
    mov ah, 2
    int 21h
    mov dl, 0ah
    mov ah, 2
    int 21h
main endp
gotoloop proc
l1:
  mov ah, 1
    int 21h
    cmp al, 13
    je end_input
    mov [si], al
    inc si
```

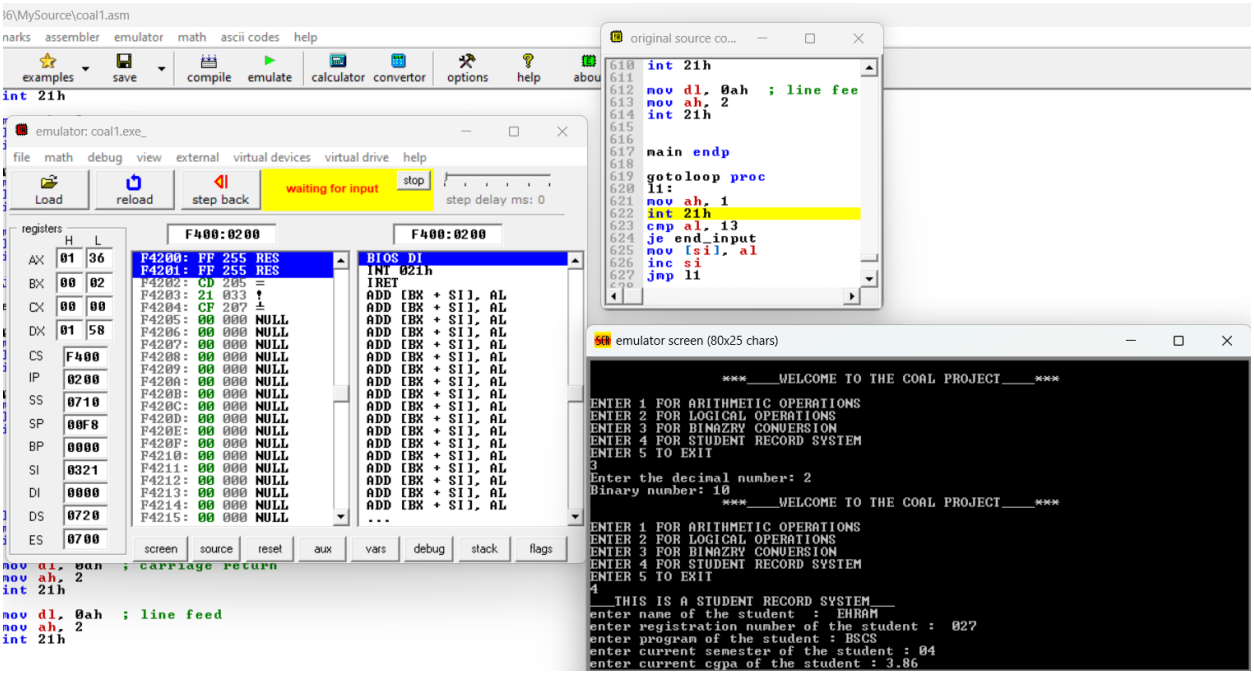
```
    jmp l1

end_input:

    mov byte ptr [si], '$'

    ret

gotoloop end

end main
```

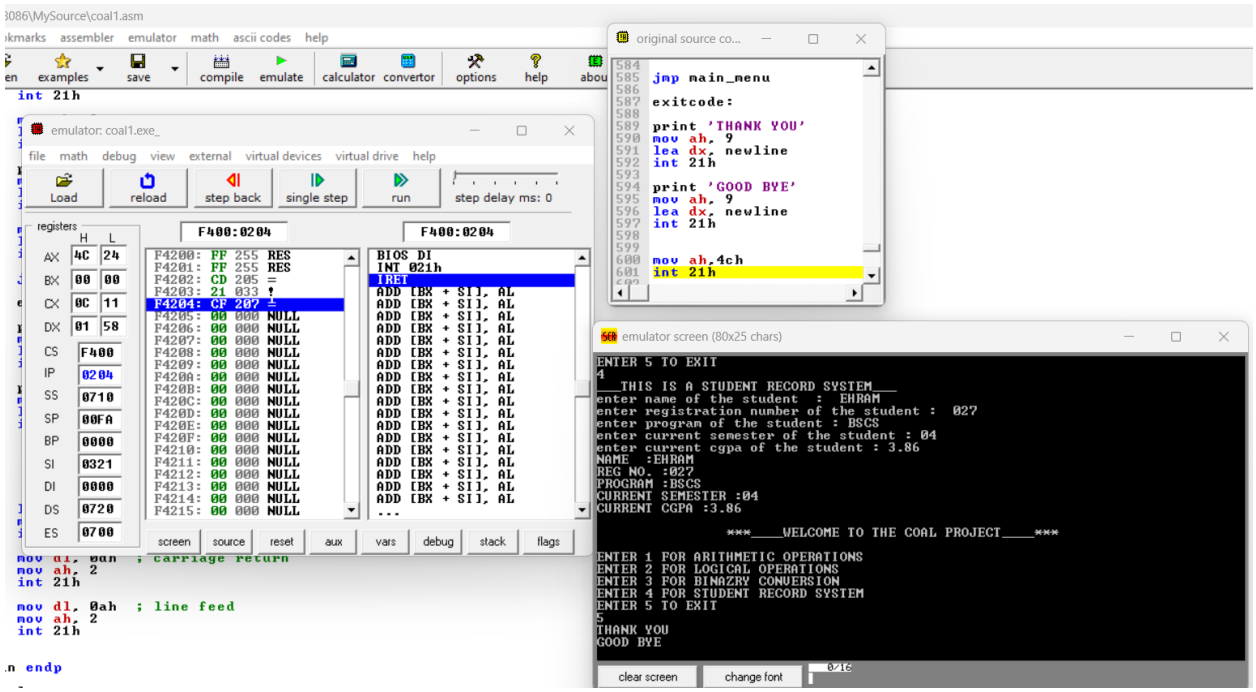
OUTPUT:



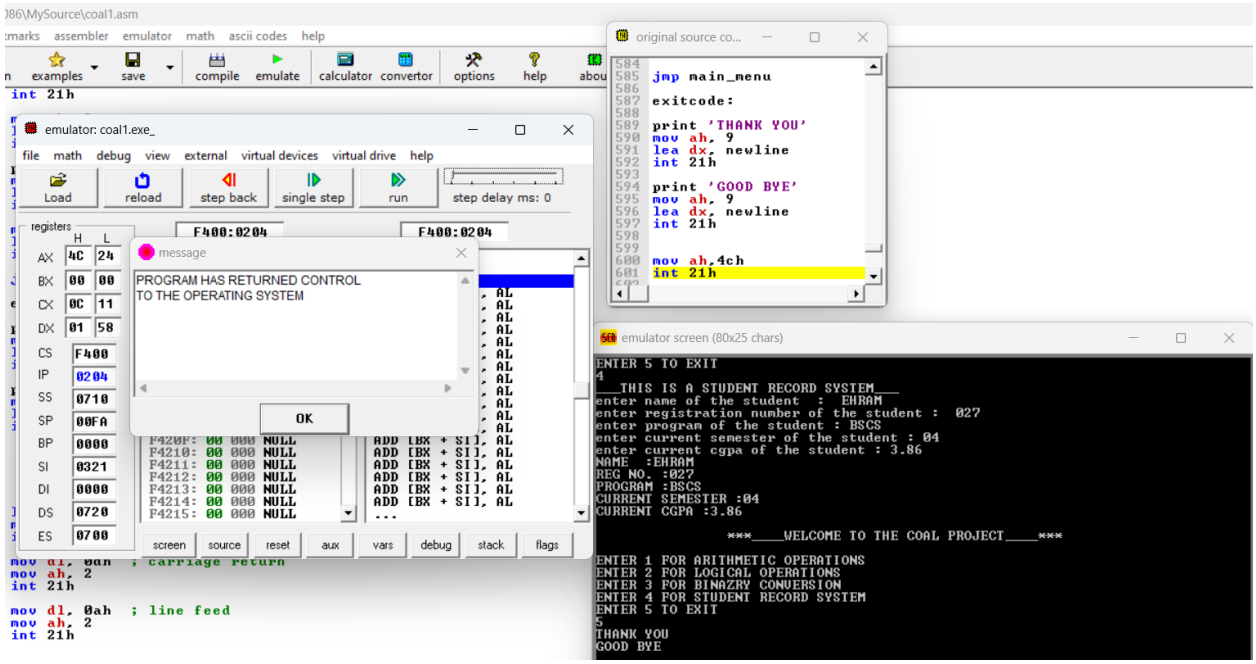



8086\MySource\coal1.asm
emulator: coal1.exe_
0759:0442
SUB AL, 030h
MOV AH, 00h
MOV [0018Bh], AX
MOV BX, 00002h
MOV CX, 00000h
MOV DX, 00000h
MOV AX, [0018Bh]
DIV BX
MOV [0018Bh], AX
PUSH DX
INC CX
CMP AX, 00000h
JNE 043Ah
MOV AH, 09h
MOV DX, 00179h
INT 021h
POP DX
ADD DL, 030h
MOV AH, 02h
INT 021h
LOOP 0450h
original source co...
int 21h
sub al, 30h
mov ah,0
mov num9, ax
mov bx, 2
mov cx, 0
mov dx,0
again:
mov ax, num9
div bx
mov num9, ax
push dx
inc cx
cmp ax, 0
jne again
emulator screen (80x25 chars)
For Exit from this portion :5
Choose Any One:5
____WELCOME TO THE COAL PROJECT____
ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
2
Enter the first number: 6
Enter the second number: 7
Select operation (1: AND, 2: OR, 3: XOR, 4: NOT): 3
Result: 1
____WELCOME TO THE COAL PROJECT____
ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
3


8086\MySource\coal1.asm
emulator: coal1.exe_
waiting for input
F400:0200
BIOS DI
INT 021h
IRET
original source co...
int 21h
mov dl, 0ah ; line fee
mov ah, 2
int 21h
main endp
gotoloop proc
l1:
mov ah, 1
int 21h
cmp al, 13
je end_input
mov [si], al
inc si
jmp l1
emulator screen (80x25 chars)
____WELCOME TO THE COAL PROJECT____
ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
3
Enter the decimal number: 2
Binary number: 10
____WELCOME TO THE COAL PROJECT____
ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
4
___THIS IS A STUDENT RECORD SYSTEM___
enter name of the student : EHRAM
enter registration number of the student : 027
enter program of the student : BSCS
enter current semester of the student : 04
enter current cgpa of the student : 3.86


emulator screen (80x25 chars)
ENTER 5 TO EXIT
4
___THIS IS A STUDENT RECORD SYSTEM___
enter name of the student : EHRAM
enter registration number of the student : 027
enter program of the student : BSCS
enter current semester of the student : 04
enter current cgpa of the student : 3.86
NAME :EHRAM
REG NO. :027
PROGRAM :BSCS
CURRENT SEMESTER :04
CURRENT CGPA :3.86
____WELCOME TO THE COAL PROJECT____
ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
5
THANK YOU
GOOD BYE

OR


message
PROGRAM HAS RETURNED CONTROL
TO THE OPERATING SYSTEM
OK
emulator screen (80x25 chars)
ENTER 5 TO EXIT
4
___THIS IS A STUDENT RECORD SYSTEM___
enter name of the student : EHRAM
enter registration number of the student : 027
enter program of the student : BSCS
enter current semester of the student : 04
enter current cgpa of the student : 3.86
NAME :EHRAM
REG NO. :027
PROGRAM :BSCS
CURRENT SEMESTER :04
CURRENT CGPA :3.86
____WELCOME TO THE COAL PROJECT____
ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
5
THANK YOU
GOOD BYE

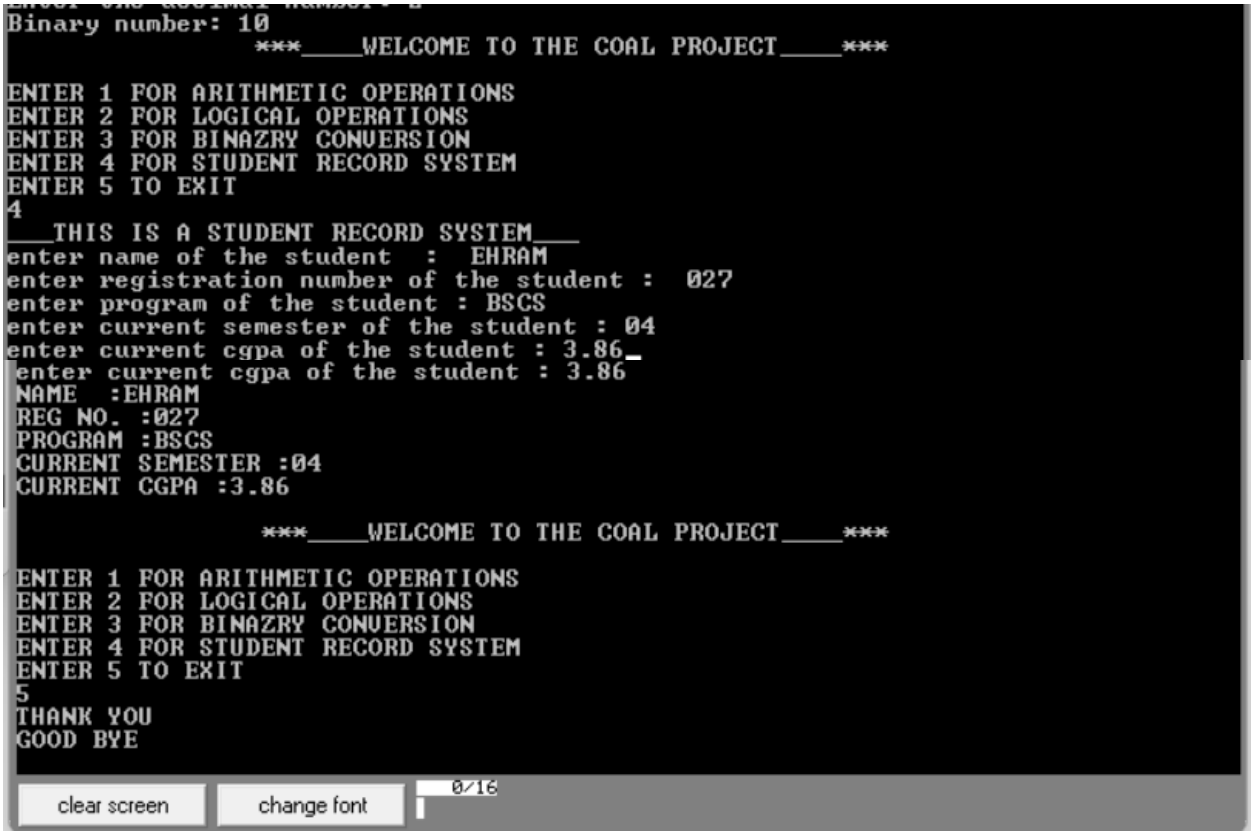
emulator screen (80x25 chars)

```
                ***____WELCOME TO THE COAL PROJECT____***

ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
1
For Add type  :1
For Sub type  :2
For Mul type  :3
For Div type  :4
For Exit from this portion :5
Choose Any One:4
```

```
Choose Any One:4

Enter 1st Number:8
Enter 2nd Number:2

The Result is:04
Choose Any One:
For Add type  :1
For Sub type  :2
For Mul type  :3
For Div type  :4
For Exit from this portion :5
```

```
For Exit from this portion :5
Choose Any One:5
                ***____WELCOME TO THE COAL PROJECT____***

ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
2

Enter the first number: 5
Enter the second number: 6
Select operation (1: AND, 2: OR, 3: XOR, 4: NOT): 3
Result: 3
                ***____WELCOME TO THE COAL PROJECT____***

ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
```

```
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
3
Enter the decimal number: 2
Binary number: 10
                ***____WELCOME TO THE COAL PROJECT____***

ENTER 1 FOR ARITHMETIC OPERATIONS
ENTER 2 FOR LOGICAL OPERATIONS
ENTER 3 FOR BINAZRY CONVERSION
ENTER 4 FOR STUDENT RECORD SYSTEM
ENTER 5 TO EXIT
```



EXPLANATION:

Main Menu: The main menu is the entry point of the program. It prints a welcome message and lists the available options. The user's input determines which section of the program will be executed.

Arithmetic Operations: In the arithmetic operations section, the user can choose to add, subtract, multiply, or divide two numbers. Each operation has a corresponding code segment that reads two input numbers from the user, performs the calculation, and displays the result.

Logical Operations: The logical operations section allows the user to perform AND, OR, XOR, and NOT operations on two numbers. The program reads the inputs and the chosen operation, then computes and displays the result.

Binary Conversion: This section converts a decimal number entered by the user into its binary equivalent. The conversion is done using a loop that repeatedly

divides the number by 2 and stores the remainders. These remainders are then displayed as the binary representation.

Student Registration System: The student registration system prompts the user to input student details, such as name, registration number, program, semester, and CGPA. These details are stored in memory and then displayed back to the user. This part demonstrates string handling and memory operations in assembly language.

CONCLUSION:

This assembly language program provides a comprehensive demonstration of basic operations and user interactions using x86 assembly language and DOS interrupts. By organizing the code into distinct sections for different functionalities, the program remains structured and easy to follow. Each section showcases different aspects of programming in assembly, such as arithmetic calculations, logical operations, binary conversion, and string handling, making it a valuable learning tool for understanding the fundamentals of low-level programming.

==========